

LAB_10.4
NLP_2403A52019
P.Navadeep

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.manifold import TSNE
```

```
# Extract vectors for selected words
words = ['king', 'queen', 'man', 'woman', 'apple', 'banana', 'car', 'truck'] # Define words here
word_vectors = []

for word in words:
    if word in model.key_to_index: # Changed model.vocab to model.key_to_index
        word_vectors.append(model[word])
    else:
        print(f"{word} not found in vocabulary.")

# Convert to NumPy array
word_vectors = np.array(word_vectors)

print("Shape of word_vectors:", word_vectors.shape)
```

Shape of word_vectors: (8, 100)

```
# Initialize t-SNE with 2 components and a perplexity value less than n_samples
tsne = TSNE(n_components=2, random_state=42, perplexity=5)

# Fit t-SNE to the word vectors and transform them
tsne_results = tsne.fit_transform(word_vectors)

print("Shape of t-SNE results:", tsne_results.shape)
print("\nFirst 5 t-SNE results:\n", tsne_results[:5])
```

Shape of t-SNE results: (8, 2)

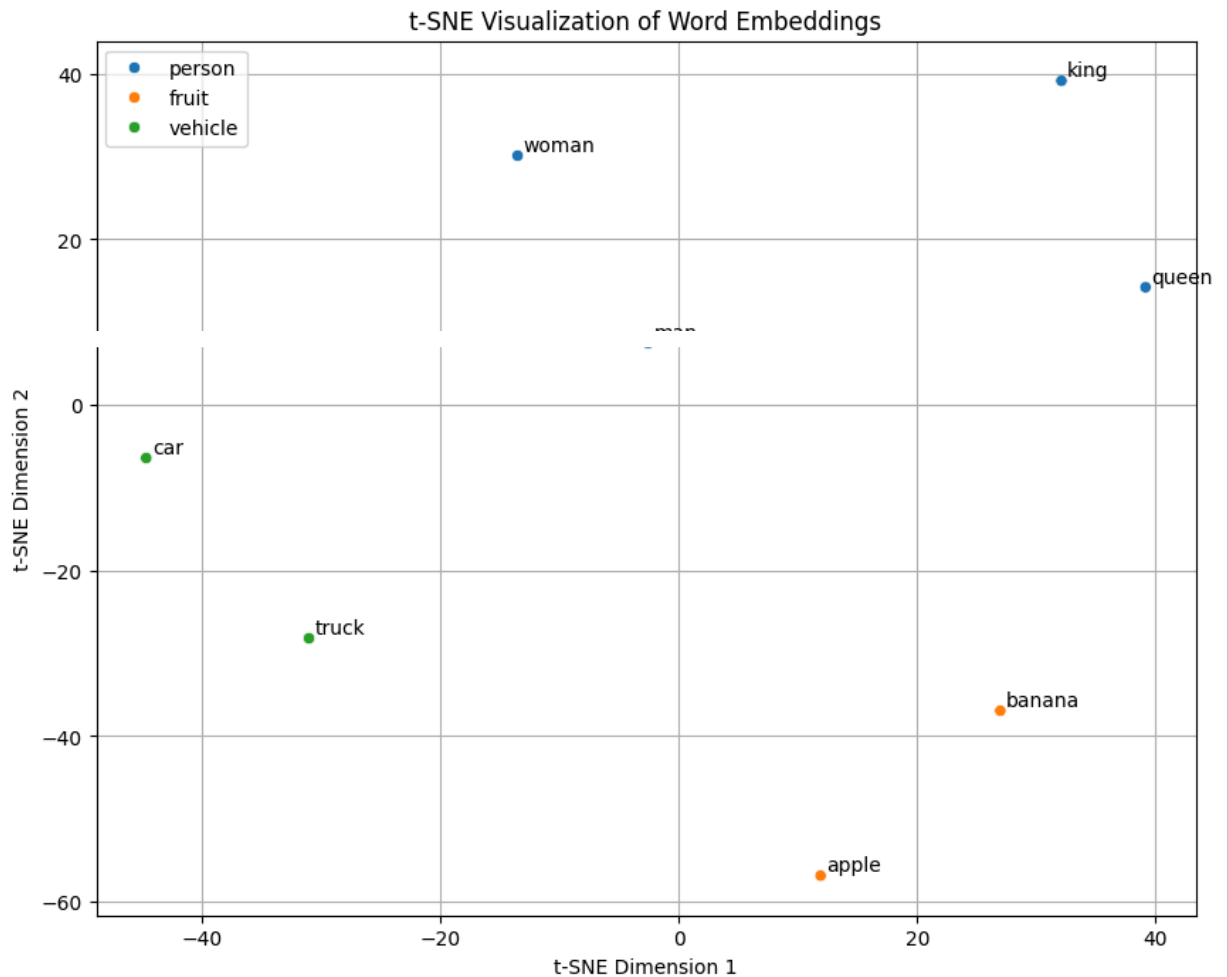
```
First 5 t-SNE results:
[[ 32.086254  39.15024 ]
 [ 39.14568   14.213357]
 [-2.6001725  7.4570756]
 [-13.548307  30.105522 ]
 [ 11.8757515 -56.846565 ]]
```

```
# STEP 6 - Plot Visualization
# Students must:
# • create scatter plot
# • annotate each point with word label
# • highlight visible clusters
# Optional: add colors per category.
```

```
# Create the scatter plot
plt.figure(figsize=(10, 8))
scatter = sns.scatterplot(
    x=tsne_results[:, 0],
    y=tsne_results[:, 1],
    hue=['person', 'person', 'person', 'person', 'fruit', 'fruit', 'vehicle', 'vehicle'], # Exam
    palette='tab10',
    legend='full'
)

# Annotate each point with the word label
```

```
for i, word in enumerate(words):  
    plt.annotate(word, (tsne_results[i, 0] + 0.5, tsne_results[i, 1] + 0.5))  
  
# Optional: Highlight visible clusters (adjust based on actual clusters)  
# For this small dataset, we'll just rely on the color coding  
  
plt.title('t-SNE Visualization of Word Embeddings')  
plt.xlabel('t-SNE Dimension 1')  
plt.ylabel('t-SNE Dimension 2')  
plt.grid(True)  
plt.show()
```



Start coding or [generate](#) with AI.

